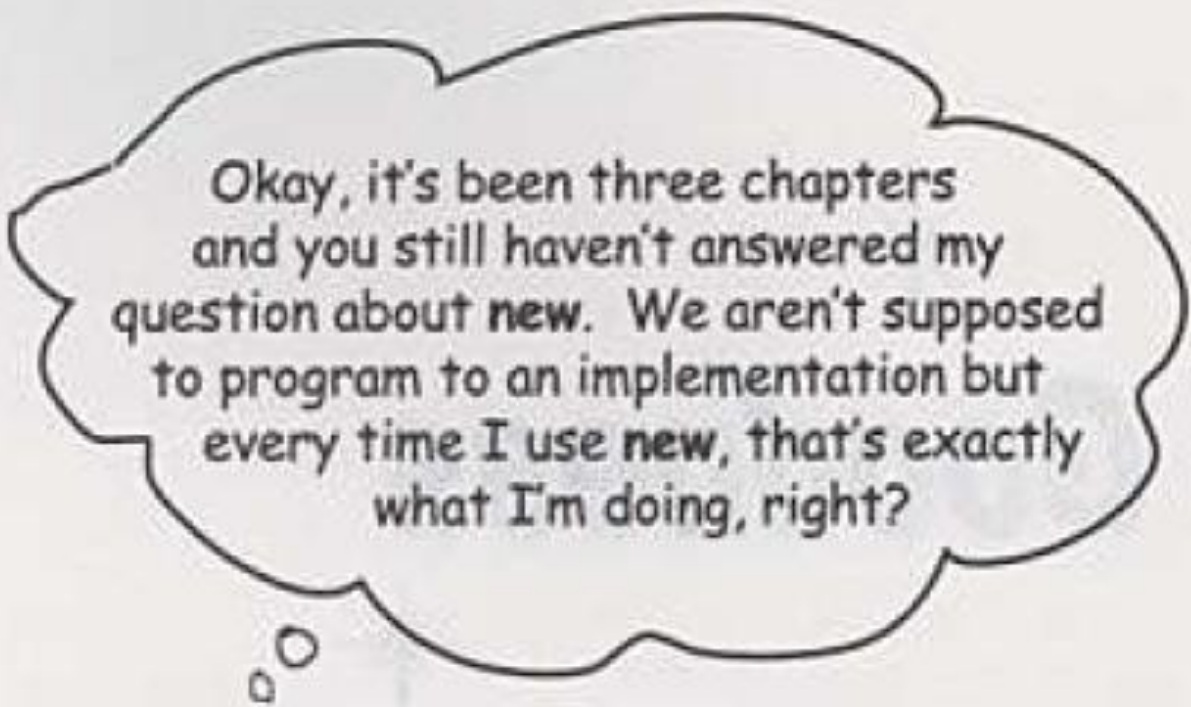


Design Patterns

Factory Method

Get ready to bake some loosely coupled OO designs. There is more to making objects than just using the `new` operator. You'll learn that instantiation is an activity that shouldn't always be done in public and can often lead to *coupling problems*. And you don't want *that*, do you? Find out how Factory Patterns can help save you from embarrassing dependencies.



Okay, it's been three chapters
and you still haven't answered my
question about `new`. We aren't supposed
to program to an implementation but
every time I use `new`, that's exactly
what I'm doing, right?

```
Duck duck = new MallardDuck();
```



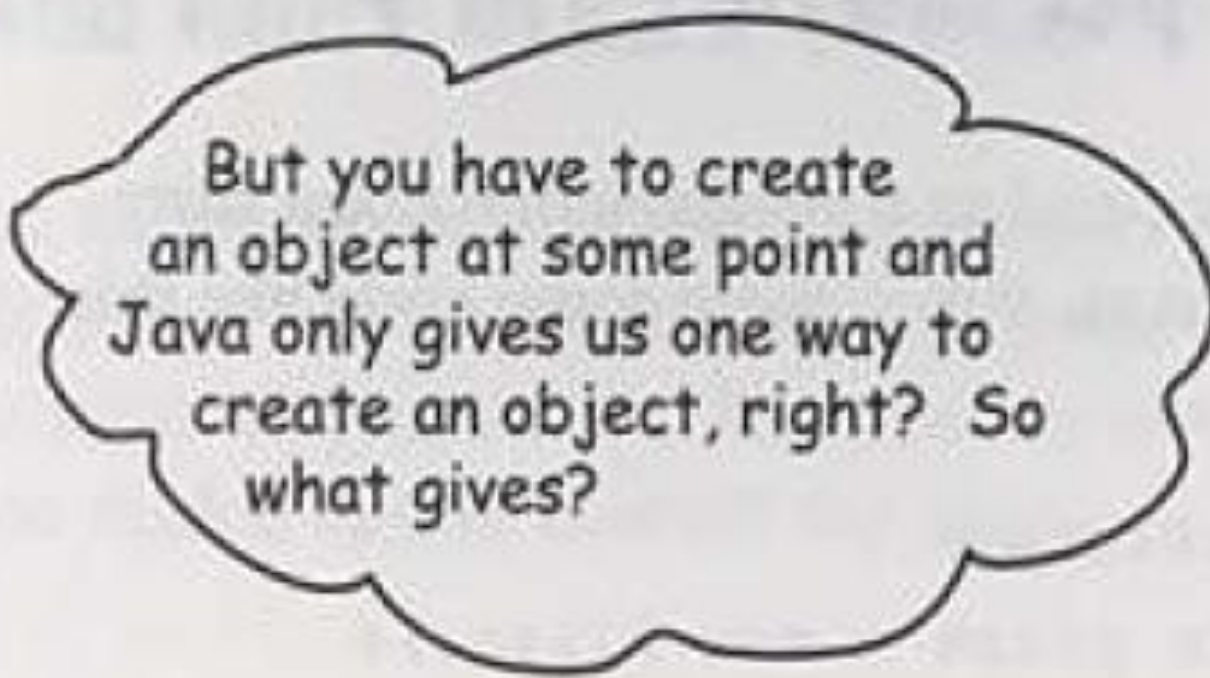
We want to use interfaces
to keep code flexible.



But we have to create an
instance of a concrete class!

```
Duck duck;
```

```
if (picnic) {  
    duck = new MallardDuck();  
} else if (hunting) {  
    duck = new DecoyDuck();  
} else if (inBathTub) {  
    duck = new RubberDuck();  
}
```



But you have to create
an object at some point and
Java only gives us one way to
create an object, right? So
what gives?



```
Pizza orderPizza() {  
    Pizza pizza = new Pizza();  
  
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
  
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("greek")) {  
        pizza = new GreekPizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    }  
}
```



```
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;  
}
```

This is what varies.
As the pizza
selection changes
over time, you'll
have to modify this
code over and over.

```
Pizza orderPizza(String type) {  
    Pizza pizza;  
  
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("greek")) {  
        pizza = new GreekPizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    } else if (type.equals("clam")) {  
        pizza = new ClamPizza();  
    } else if (type.equals("veggie")) {  
        pizza = new eggiePizza();  
    }  
}
```

This code is NOT closed
for modification. If
the Pizza Shop changes
its pizza offerings, we
have to get into this
code and modify it.

```
Pizza orderPizza(String type) {
```

```
    Pizza pizza;
```

```
    pizza.prepare();
```

```
    pizza.bake();
```

```
    pizza.cut();
```

```
    pizza.box();
```

```
    return pizza;
```

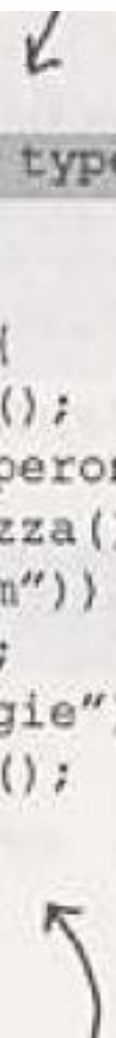
```
}
```

What

```
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    } else if (type.equals("clam")) {  
        pizza = new ClamPizza();  
    } else if (type.equals("veggie")) {  
        pizza = new VeggiePizza();  
    }  
}
```

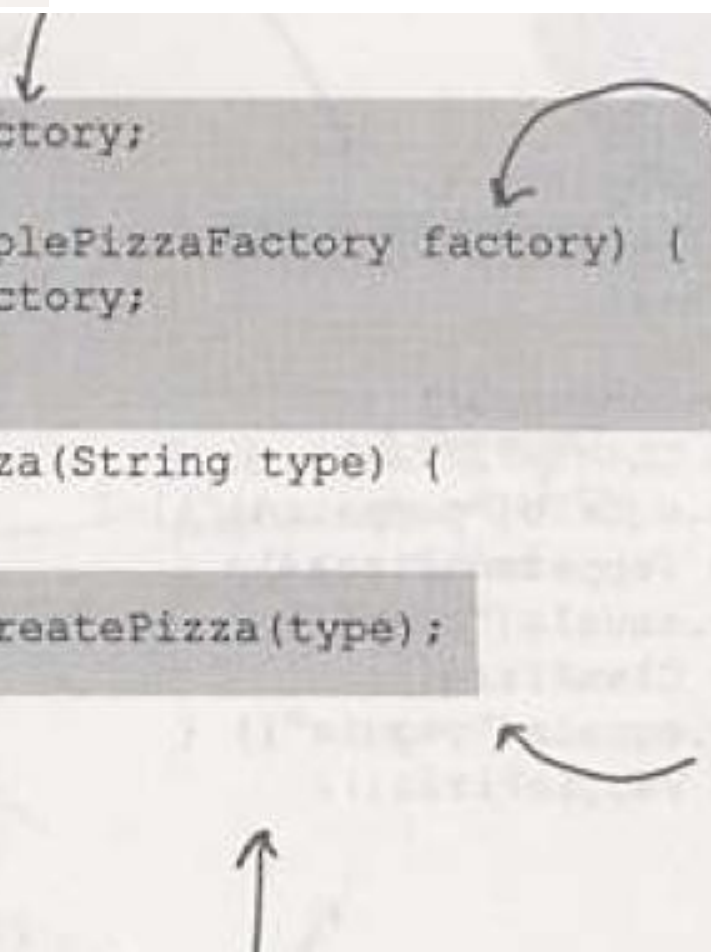


```
public class SimplePizzaFactory {  
    public Pizza createPizza(String type) {  
        Pizza pizza = null;  
  
        if (type.equals("cheese")) {  
            pizza = new CheesePizza();  
        } else if (type.equals("pepperoni")) {  
            pizza = new PepperoniPizza();  
        } else if (type.equals("clam")) {  
            pizza = new ClamPizza();  
        } else if (type.equals("veggie")) {  
            pizza = new VeggiePizza();  
        }  
        return pizza;  
    }  
}
```



Reworking the PizzaStore class

```
public class PizzaStore {  
    SimplePizzaFactory factory;  
  
    public PizzaStore(SimplePizzaFactory factory) {  
        this.factory = factory;  
    }  
  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
  
        pizza = factory.createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
        return pizza;  
    }  
  
    // other methods here  
}
```

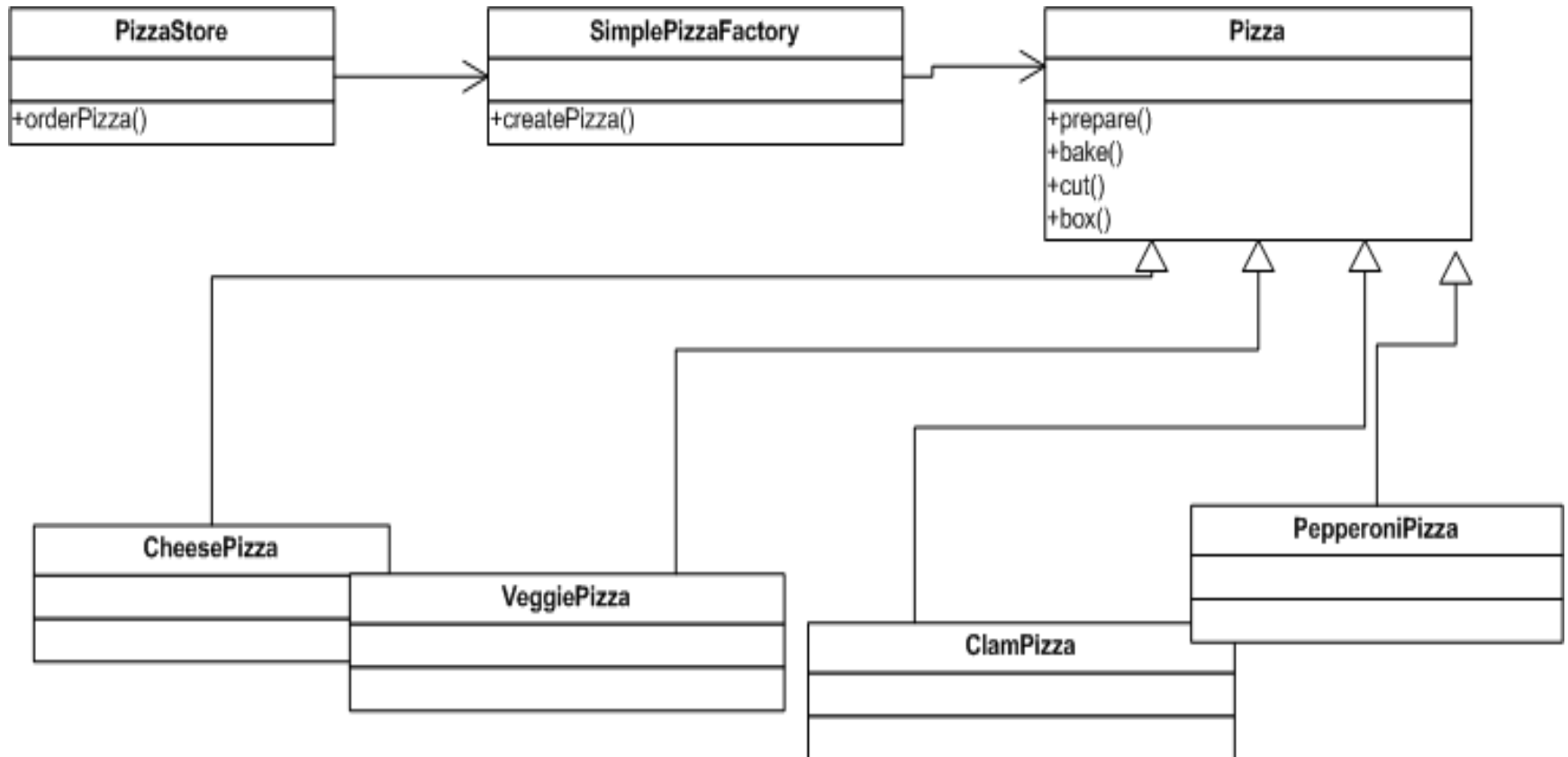


Notice that we've replaced the new operator with a create method on the factory object. No more concrete instantiations here!



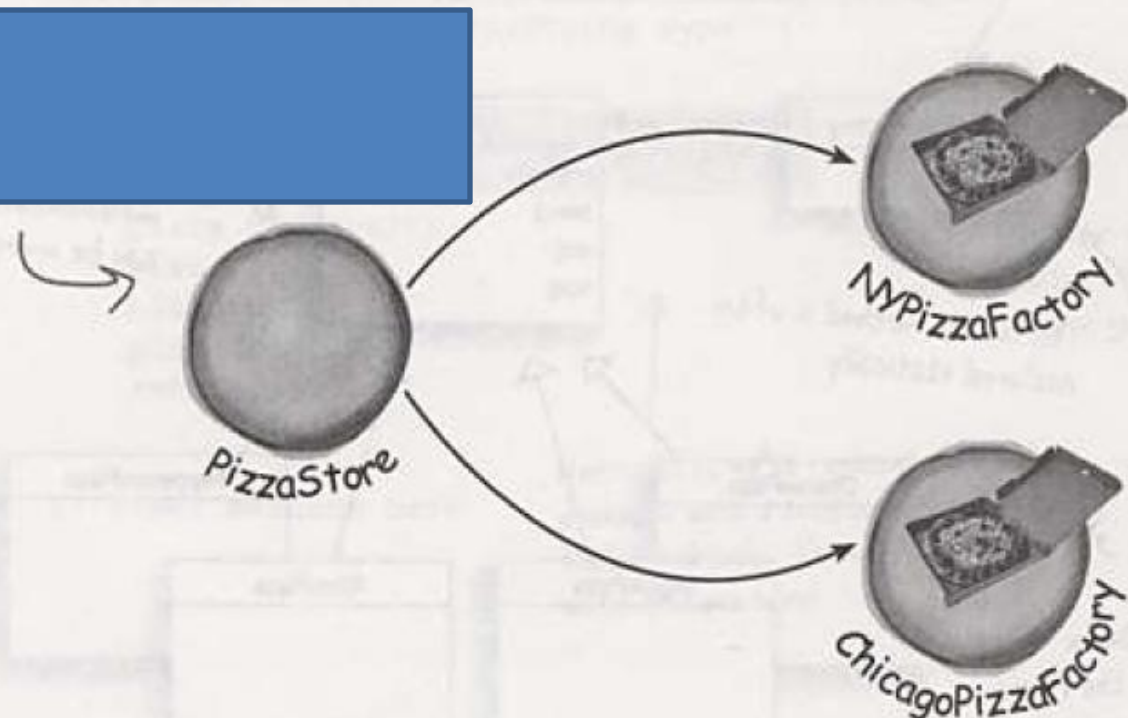
Pattern
Honorable
Mention

The Simple Factory



Franchising the pizza store

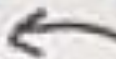
You want all the franchise pizza stores to leverage your PizzaStore code, so the pizzas are prepared in the same way.



One franchise wants a factory that makes NY style pizzas: thin crust, tasty sauce and just a little cheese.

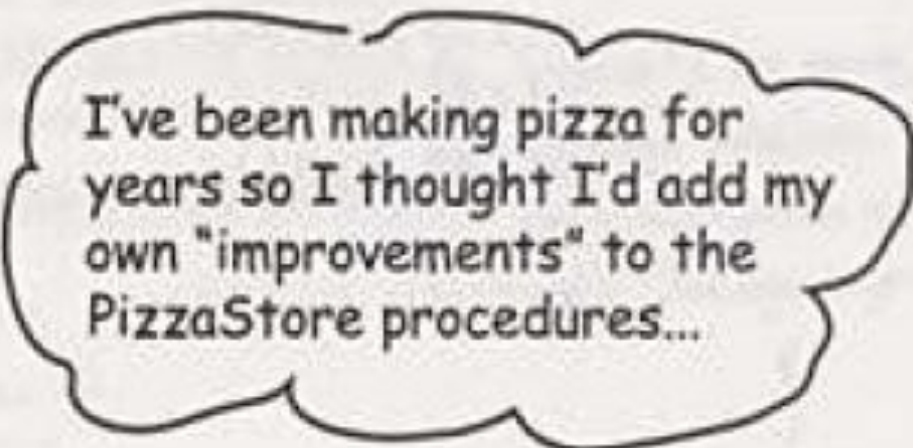
Another franchise wants a factory that makes Chicago style pizzas; their customers like pizzas with thick crust, rich sauce, and tons of cheese.

```
NYPizzaFactory nyFactory = new NYPizzaFactory();  
PizzaStore nyStore = new PizzaStore(nyFactory);  
nyStore.order("Veggie");
```



```
ChicagoPizzaFactory chicagoFactory = new ChicagoPizzaFactory();  
PizzaStore chicagoStore = new PizzaStore(chicagoFactory);  
chicagoStore.order("Veggie");
```

But you'd like a little more quality control...



I've been making pizza for years so I thought I'd add my own "improvements" to the PizzaStore procedures...

Not what you want in a good franchise. You do NOT want to know what he puts on his pizzas.



```
public abstract class PizzaStore {
```

```
    public Pizza orderPizza(String type) {  
        Pizza pizza;
```

```
        pizza = createPizza(type);
```

```
        pizza.prepare();
```

```
        pizza.bake();
```

```
        pizza.cut();
```

```
        pizza.box();
```

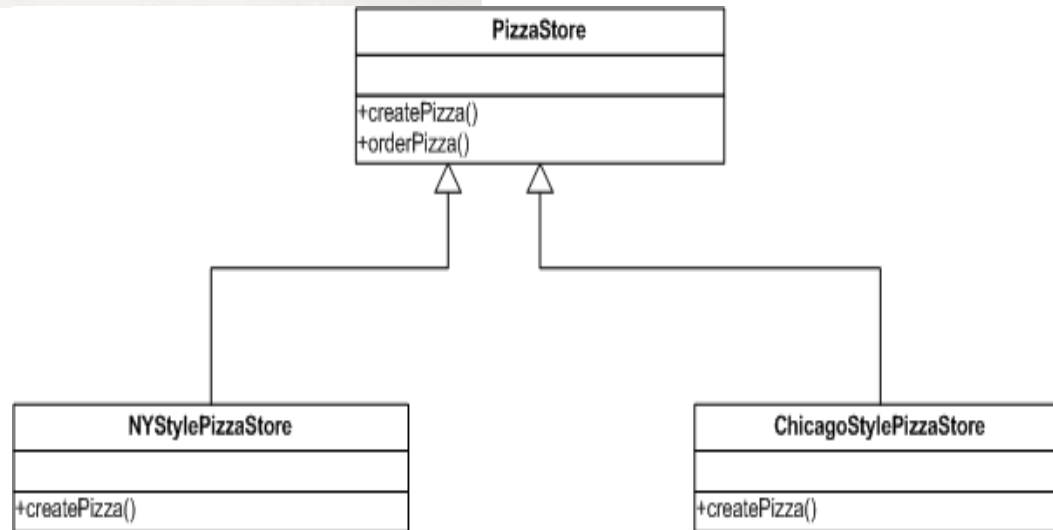
```
        return pizza;
```

```
    }
```

```
    abstract createPizza(String type);
```

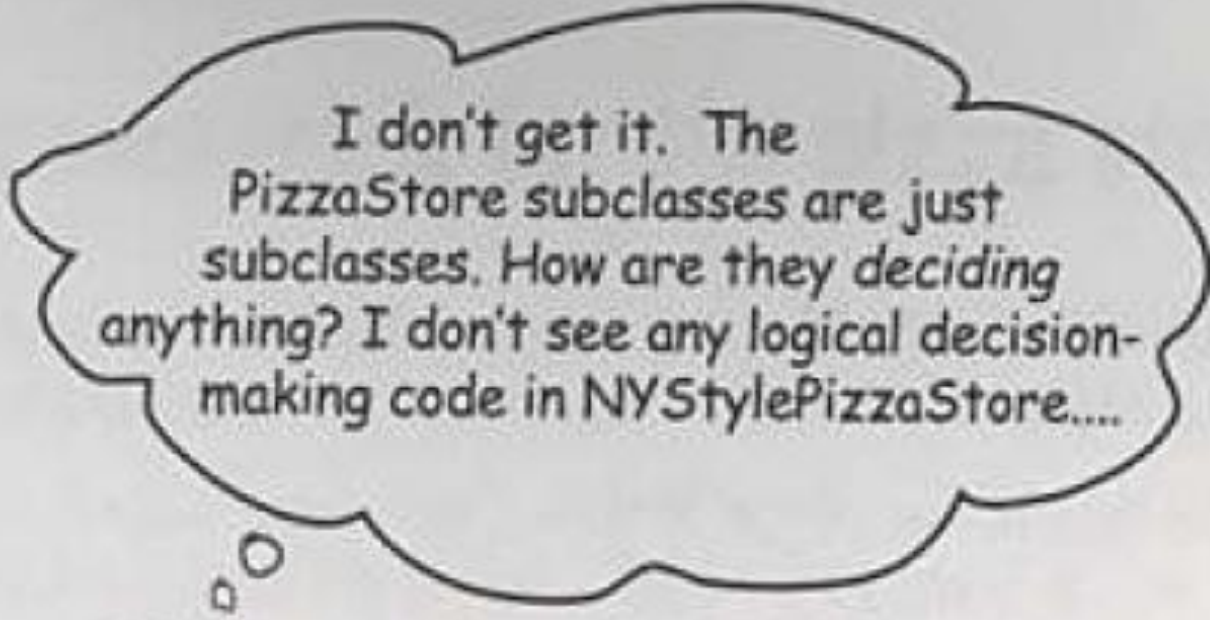
```
}
```

Allowing the subclasses to decide

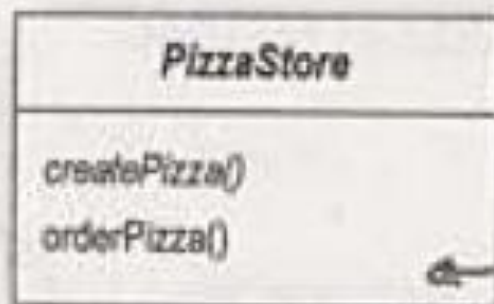


```
public Pizza createPizza(type) {
    if (type.equals("cheese")) {
        pizza = new NYStyleCheesePizza();
    } else if (type.equals("pepperoni")) {
        pizza = new NYStylePepperoniPizza();
    } else if (type.equals("clam")) {
        pizza = new NYStyleClamPizza();
    } else if (type.equals("veggie")) {
        pizza = new NYStyleVeggiePizza();
    }
}
```

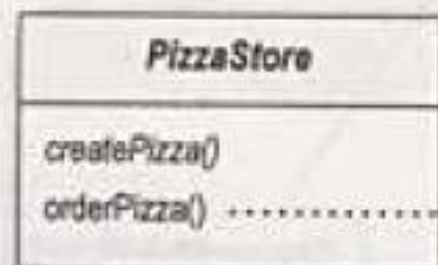
```
public Pizza createPizza(type) {
    if (type.equals("cheese")) {
        pizza = new ChicagoStyleCheesePizza();
    } else if (type.equals("pepperoni")) {
        pizza = new ChicagoStylePepperoniPizza();
    } else if (type.equals("clam")) {
        pizza = new ChicagoStyleClamPizza();
    } else if (type.equals("veggie")) {
        pizza = new ChicagoStyleVeggiePizza();
    }
}
```



I don't get it. The
PizzaStore subclasses are just
subclasses. How are they *deciding*
anything? I don't see any logical decision-
making code in NYStylePizzaStore....



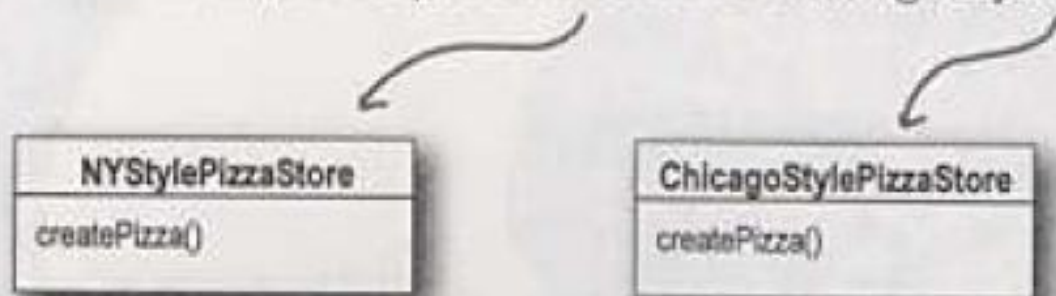
`orderPizza()` is defined in the abstract `PizzaStore`, not the subclasses. So, the method has no idea which subclass is actually running the code and making the pizzas.



```
pizza = createPizza();
pizza.prepare();
pizza.bake();
pizza.cut();
pizza.box();
```

`orderPizza()` calls `createPizza()` to actually get a pizza object. But which kind of pizza will it get? The `orderPizza()` method can't decide; it doesn't know how. So who does decide?

When `orderPizza()` calls `createPizza()`, one of your subclasses will be called into action to create a pizza. Which kind of pizza will be made? Well, that's decided by the choice of pizza store you order from, `NYStylePizzaStore` or `ChicagoStylePizzaStore`.



Let's make a PizzaStore

```
public class NYPizzaStore extends PizzaStore {  
    Pizza createPizza(String item) {  
        if (item.equals("cheese")) {  
            return new NYStyleCheesePizza();  
        } else if (item.equals("veggie")) {  
            return new NYStyleVeggiePizza();  
        } else if (item.equals("clam")) {  
            return new NYStyleClamPizza();  
        } else if (item.equals("pepperoni")) {  
            return new NYStylePepperoniPizza();  
        } else return null;  
    }  
}
```

`createPizza()` returns a `Pizza`, and the subclass is fully responsible for which concrete `Pizza` it instantiates

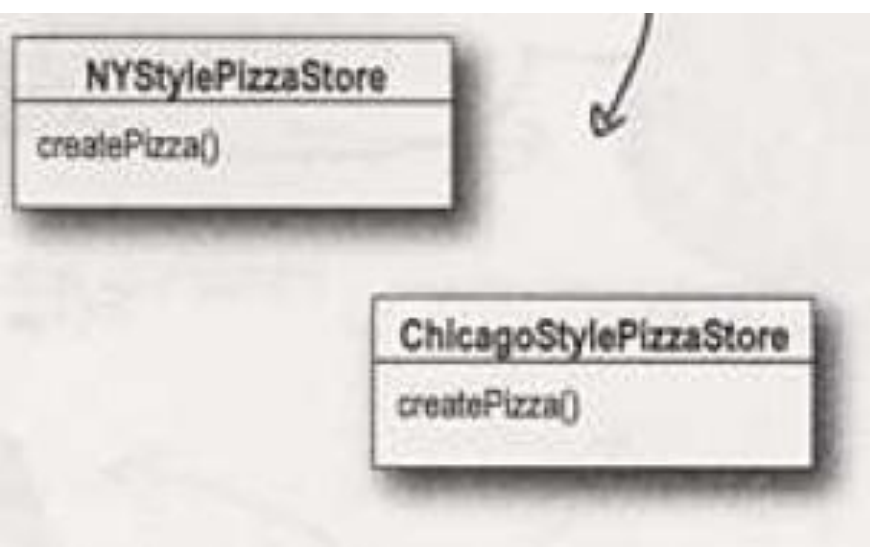
The `NYPizzaStore` extends `PizzaStore`, so it inherits the `orderPizza()` method (among others).

We've got to implement `createPizza()`, since it is abstract in `PizzaStore`.

* Note that the `orderPizza()` method in the superclass has no clue which `Pizza` we are creating; it just knows it can prepare, bake, cut, and box it!

Declaring a factory method

```
public abstract class PizzaStore {  
  
    public Pizza orderPizza(String type) {  
        Pizza pizza;  
  
        pizza = createPizza(type);  
  
        pizza.prepare();  
        pizza.bake();  
        pizza.cut();  
        pizza.box();  
  
        return pizza;  
    }  
  
    protected abstract Pizza createPizza(String type);  
  
    // other methods here
```



The subclasses of `PizzaStore` handle object instantiation for us in the `createPizza()` method.

All the responsibility for instantiating Pizzas has been moved into a method that acts as a factory.



Code Up Close

A factory method handles object creation and encapsulates it in a subclass. This decouples the client code in the superclass from the object creation code in the subclass.

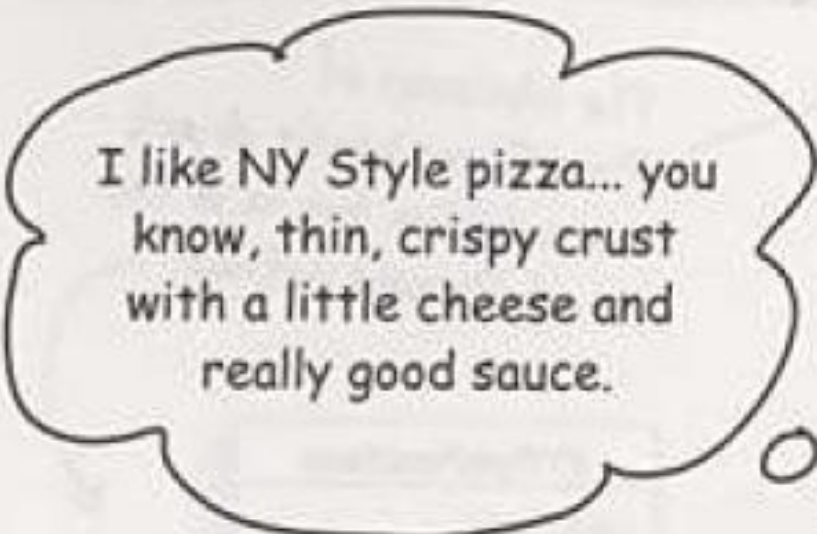
A factory method may be parameterized (or not) to select among several variations of a product.

abstract Product factoryMethod(String type)

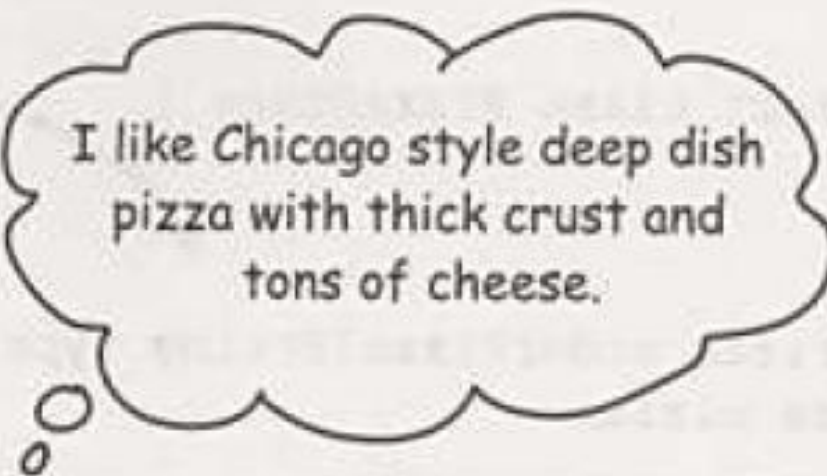
A factory method is abstract so the subclasses are counted on to handle object creation.

A factory method returns a Product that is typically used within methods defined in the superclass.

A factory method isolates the client (the code in the superclass, like `orderPizza()`) from knowing what kind of concrete Product is actually created.



I like NY Style pizza... you know, thin, crispy crust with a little cheese and really good sauce.



I like Chicago style deep dish pizza with thick crust and tons of cheese.

So how do they order?

- 1 First, Joel and Ethan need an instance of a `PizzaStore`. Joel needs to instantiate a `ChicagoPizzaStore` and Ethan needs a `NYPizzaStore`.
- 2 With a `PizzaStore` in hand, both Ethan and Joel call the `orderPizza()` method and pass in the type of pizza they want (cheese, veggie, and so on).
- 3 To create the pizzas, the `createPizza()` method is called, which is defined in the two subclasses `NYPizzaStore` and `ChicagoPizzaStore`. As we defined them, the `NYPizzaStore` instantiates a NY style pizza, and the `ChicagoPizzaStore` instantiates Chicago style pizza. In either case, the `Pizza` is returned to the `orderPizza()` method.
- 4 The `orderPizza()` method has no idea what kind of pizza was created, but it knows it is a pizza and it prepares, bakes, cuts, and boxes it for Ethan and Joel.

1 Let's follow Ethan's order: first we need a NY PizzaStore:

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```



2 Now that we have a store, we can take an order:

```
nyPizzaStore.orderPizza("cheese");
```

3 The orderPizza() method then calls the createPizza() method:

```
Pizza pizza = createPizza("cheese");
```

Remember, createPizza(), the factory method, is implemented in the subclass. In this case it returns a NY Cheese Pizza.

4

Finally we have the unprepared pizza in hand and the `orderPizza()` method finishes preparing it:

```
pizza.prepare();  
pizza.bake();  
pizza.cut();  
pizza.box();
```

All of these methods are defined in the specific pizza returned from the factory method `createPizza()`, defined in the `NYPizzaStore`.

We're just missing one thing: PIZZA!

Each Pizza has a name, a type of dough, a type of sauce, and a set of toppings.

```
public abstract class Pizza {  
    String name;  
    String dough;  
    String sauce;  
    ArrayList toppings = new ArrayList();  
}
```



```
void prepare() {  
    System.out.println("Preparing " + name);  
    System.out.println("Tossing dough...");  
    System.out.println("Adding sauce...");  
    System.out.println("Adding toppings: ");  
    for (int i = 0; i < toppings.size(); i++) {  
        System.out.println("    " + toppings.get(i));  
    }  
}
```

The abstract class provides some basic defaults for baking, cutting and boxing.

```
void bake() {  
    System.out.println("Bake for 25 minutes at 350");  
}  
  
void cut() {  
    System.out.println("Cutting the pizza into diagonal slices");  
}  
  
void box() {  
    System.out.println("Place pizza in official PizzaStore box");  
}  
  
public String getName() {  
    return name;  
}  
}
```

Now we just need some concrete subclasses... how about defining New York and Chicago style cheese pizzas?

```
public class NYStyleCheesePizza extends Pizza {  
    public NYStyleCheesePizza() {  
        name = "NY Style Sauce and Cheese Pizza";  
        dough = "Thin Crust Dough";  
        sauce = "Marinara Sauce";  
  
        toppings.add("Grated Reggiano Cheese");  
    }  
}
```

```
public class ChicagoStyleCheesePizza extends Pizza {
    public ChicagoStyleCheesePizza() {
        name = "Chicago Style Deep Dish Cheese Pizza";
        dough = "Extra Thick Crust Dough";
        sauce = "Plum Tomato Sauce";

        toppings.add("Shredded Mozzarella Cheese");
    }

    void cut() {
        System.out.println("Cutting the pizza into square slices");
    }
}
```

with extra t

The Chic
dish pizz
mozzare

The Chicago style pizza also overrides the cut() method so that the pieces are cut into squares.

You've waited long enough, time for some pizzas!

```
public class PizzaTestDrive {  
    public static void main(String[] args) {  
        PizzaStore nyStore = new NYPizzaStore();  
        PizzaStore chicagoStore = new ChicagoPizzaStore();  
  
        Pizza pizza = nyStore.orderPizza("cheese");  
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");  
  
        pizza = chicagoStore.orderPizza("cheese");  
        System.out.println("Joel ordered a " + pizza.getName() + "\n");  
    }  
}
```

First we create different stores
Then to

File Edit Window Help YouWantMootzOnThatPizza?

```
%java PizzaTestDrive
```

```
Preparing NY Style Sauce and Cheese Pizza
```

```
Tossing dough...
```

```
Adding sauce...
```

```
Adding toppings:
```

```
    Grated Regiano cheese
```

```
Bake for 25 minutes at 350
```

```
Cutting the pizza into diagonal slices
```

```
Place pizza in official PizzaStore box
```

```
Ethan ordered a NY Style Sauce and Cheese Pizza
```

```
Preparing Chicago Style Deep Dish Cheese Pizza
```

```
Tossing dough...
```

```
Adding sauce...
```

```
Adding toppings:
```

```
    Shredded Mozzarella Cheese
```

```
Bake for 25 minutes at 350
```

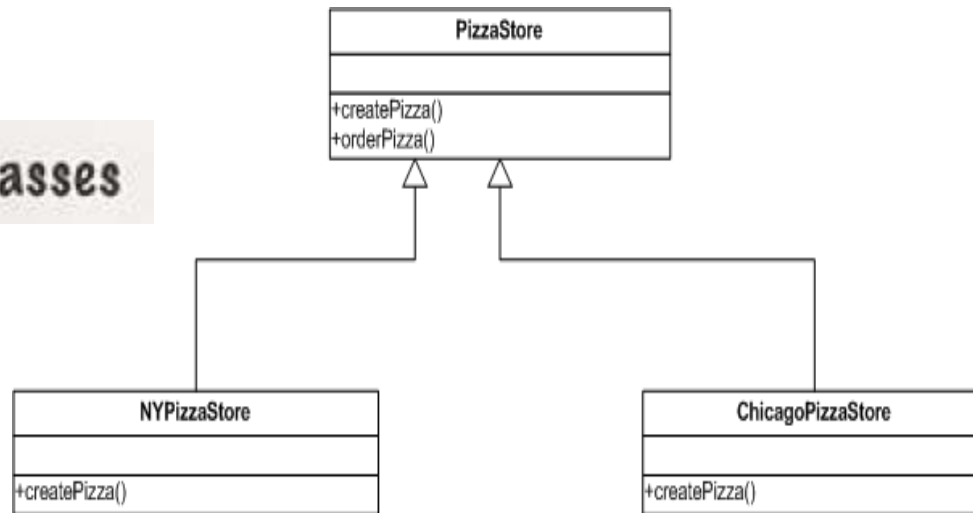
```
Cutting the pizza into square slices
```

```
Place pizza in official PizzaStore box
```

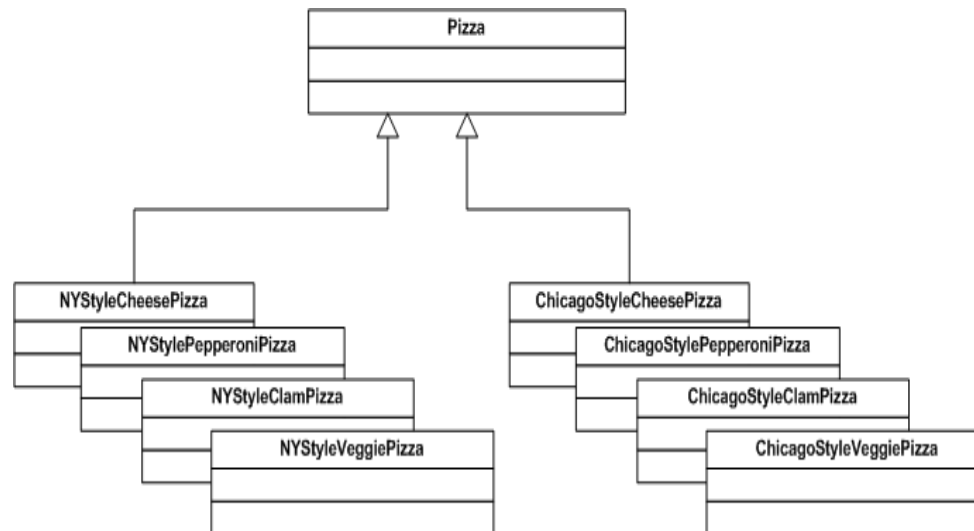
```
Joel ordered a Chicago Style Deep Dish Cheese Pizza
```

It's finally time to meet the Factory Method Pattern

The Creator classes



The Product classes

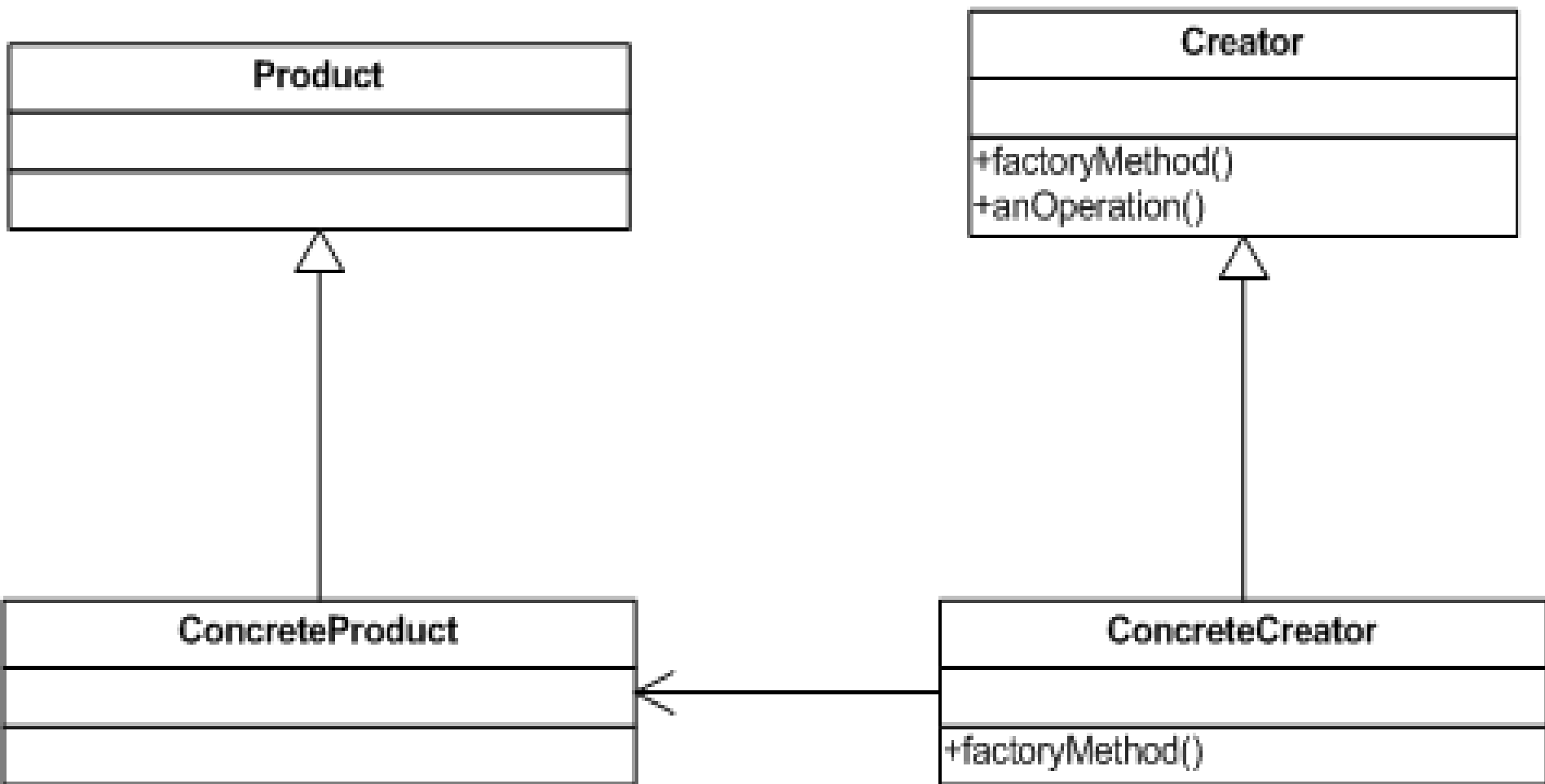


parallel class hierarchies

Notice how these class hierarchies are parallel: both have abstract classes that are extended by concrete classes, which know about specific implementations for NY and Chicago.

The factory method is the key to encapsulating this knowledge.

The Factory Method Pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.



Q: What's the advantage of the Factory Method Pattern when you only have one ConcreteCreator?

A: The Factory Method Pattern is useful if you've only got one concrete creator because you are decoupling the implementation of the product from its use. If you add additional products or change a product's implementation, it will not affect your Creator (because the Creator is not tightly coupled to any ConcreteProduct).

Q: Are the factory method and the Creator always abstract?

A: No, you can define a default factory method to produce some concrete product. Then you always have a means of creating products even if there are no subclasses of the Creator.

Q: Would it be correct to say that our NY and Chicago stores are implemented using Simple Factory? They look just like it.

A: They're similar, but used in different ways. Even though the implementation of each concrete store looks a lot like the SimplePizzaFactory, remember that the concrete stores are extending a class which has defined createPizza() as an abstract method. It is up to each store to define the behavior of the createPizza() method. In Simple Factory, the factory is another object that is composed with the PizzaStore.